pre

p1

Hello, everyone,
Our topic today is titled Fox and Dinner: Solving Graph Matching with Max-Flow. It's based on a Codeforces problem that combines ideas from number theory and graph algorithms.

p2

Here's a quick overview of today's presentation.
We'll start with Part 1, where I'll introduce the problem background and give a clear definition of what we're trying to solve.
Part 2, we'll walk through the intuition behind the algorithm—the main idea and why it works.
Then, in Part 3, I'll present the step-by-step algorithm and analyze its time complexity.
Finally, Part 4 will include a proof of correctness and a deeper time analysis, to ensure our approach is both valid and efficient.

p3

Let me introduce the problem setting.
In this scenario, Fox Ciel is organizing a dinner party in the Prime Kingdom. There are n foxes, and each fox has a positive integer age. They all need to sit at one or more **round tables**.
Now, there are a few constraints we must follow:

- Each fox must sit at **exactly one** table.
- Each table must seat **at least 3 foxes**.
- Foxes at each table are seated **in a circle**, so the first and last fox are also considered adjacent.
- For **every pair of adjacent foxes**, the **sum of their ages must be a prime number**.
  Finally, our goal is to determine:
  Can we partition the foxes into tables that satisfy all these constraints?
  If yes, we should print the seating arrangement. If not, we print **"Impossible"**.

p6

Before diving into the main algorithm, we define a simple helper function called IsPrime(x) to check if a number is prime.
The logic is straightforward:

- If x is less than 2, return false.
- Otherwise, iterate from 2 up to the square root of x.
- If any number divides x evenly, it's not a prime.
- If none do, we return true.
  This function will be used frequently, especially when checking adjacent fox age sums in our problem.
  The runtime of this function is **O(√x)**, since we only check up to the square root.

p7

Here is the main algorithm used to solve the Fox Dinner problem.
We first divide the foxes into two sets: **ODD** and **EVEN**, based on the parity of their ages.
If the sizes of these two sets don't match, we immediately return "Impossible", since we need to form complete cycles.
Then we construct a bipartite graph between ODD and EVEN foxes.

If the sum of their ages is prime, we add an edge between them with capacity 1.
Next, we connect each **odd node** to a source s, and each **even node** to a sink t, both with capacity 2.
We then compute the **maximum matching** using the Ford-Fulkerson algorithm.
If the maximum flow is less than n, that means we can't pair up all the foxes, and we return "Impossible".
Otherwise, we extract the cycles by collecting edges with positive flow and perform DFS to recover actual seating arrangements.
Finally, we return all valid cycles as the solution.
The overall **runtime** is
$O(n^2\sqrt{m} + n^3)$,
where n is the number of foxes and m is the largest age value. Given that $n \leq 200$ and $m \leq 10^4$, this is efficient enough.